

Methylation

Katherine I. Cooper

2026-05-01

Methylation detection

Please run all chunks here and move to the parent file afterwards. This file contains **libraries** and a **function**.

```
## Warning: package 'ggplot2' was built under R version 4.5.3
```

CG dinucleotide counter:

The following code cell contains a function that automatically compiles CG dinucleotides from a reference genome. To view the code, uncollapse the cell; however, this is non-essential for running the function.

To load the function, please click the green arrow on the right-side of the code cell. This will load the function into your environment. It is called: countCpG.

```
countCpG <- function(df, GeneIDs, flankRegion, species){  
  
  # Set-up for the species specific portion  
  
  #Which packages are needed  
  message(paste("Loading packages for", species, "."))  
  
  Species.pkgs <- switch(species,  
    "Hs" = c("BSgenome.Hsapiens.UCSC.hg38",  
             "org.Hs.eg.db"),  
    "Mm" = c("BSgenome.Mmusculus.UCSC.mm39",  
             "org.Mm.eg.db"),  
    "Gg" = c("BSgenome.Ggallus.UCSC.galGal5",  
             "org.Gg.eg.db"),  
    stop("Invalid species choice. Supported options: Homo sapiens (Hs), Mus musculus (Mm), Gallus gallus  
    )  
  
  sapply(Species.pkgs, library, character.only = T)  
  
  #Assigning the ensembl  
  
  ensembl <- switch(species,  
    "Hs" = useMart("ensembl",  
                   dataset = "hsapiens_gene_ensembl",  
                   host = "https://may2025.archive.ensembl.org"),  
    "Mm" = useMart("ensembl",  
                   dataset = "mmusculus_gene_ensembl",
```

```

        host = "https://dec2021.archive.ensembl.org"),
"Gg" = useMart("ensembl",
              dataset = "ggallus_gene_ensembl",
              host = "https://dec2021.archive.ensembl.org"),
stop("Invalid species choice. Supported options: Homo sapiens (Hs), Mus musculus (Mm), Gallus gal.

BSgenome.sp <- switch(species,
  "Hs" = BSgenome.Hsapiens.UCSC.hg38,
  "Mm" = BSgenome.Mmusculus.UCSC.mm39,
  "Gg" = BSgenome.Ggallus.UCSC.galGal5,
  stop("Unknown BSgenome object")
)

genes = df[[GeneIDs]]

#Mapping the ensembl names to the symbols

#Getting which ensembl db

pkg.map <- c(
  "Mm" = "org.Mm.eg.db",
  "Hs" = "org.Hs.eg.db",
  "Gg" = "org.Gg.eg.db"
)

org.db <- get(pkg.map[[species]])

Ensembl_names <- mapIds(org.db,
  keys = genes, keytype = "SYMBOL", column = "ENSEMBL")

##loop set-ups

#Empty vector for the coords
coords <- list()

#Flanking regions
flanks <- flankRegion

#Counter for final df
counter = 1

##Loop 1: Assigning TSS coordinate
for (n in genes) {

  #Message for which gene

  message(paste0("(", counter, "/", length(Ensembl_names), ") Processing: ", n))

  #Checking for NA

  if(is.na(Ensembl_names[n]) || is.null(Ensembl_names[n])) {
    message(paste("Unknown gene SYMBOL ID for", n, "Skipping gene."))
  }
}

```

```

    next
  }

  #Assigning the emsembl for the loop
  Ensembl_id <- Ensembl_names[n]

  #Getting the TSS coordinate
  tss_info <- getBM(
    filters = "ensembl_gene_id",
    values = Ensembl_id,
    attributes = c("chromosome_name", "transcription_start_site", "strand"),
    mart = ensembl
  )

  #Skipping no TSS
  if(is.null(tss_info) || nrow(tss_info) == 0) {
    message(paste("Skipping gene:", n, "-No TSS information found in BioMart"))
    next
  }

  #Assigning the TSS

  tss_info <- tss_info[1, ]

  ##Loop 2: Finding CGs in the gene per each flanking regions
  for (i in flanks) {

    #Adding/subtracting flanks
    start_coord <- tss_info$transcription_start_site - i
    end_coord <- tss_info$transcription_start_site + i

    #Ensuring the start/stop are not flipped because of - strand
    if (start_coord > end_coord) {
      temp <- start_coord
      start_coord <- end_coord
      end_coord <- temp
    }

    #Pulling the DNA sequence info given the start/stop region
    tss_range <- GRanges(
      seqnames = paste0("chr", tss_info$chromosome_name),
      ranges = IRanges(start = start_coord, end = end_coord),
      strand = ifelse(tss_info$strand == 1, "+", "-")
    )

    #Getting the actual sequence
    tss_sequunce <- getSeq(BSgenome.sp, tss_range)

    #Counting CG dinucleotides
    CpG_count <- str_count(as.character(tss_sequunce), "CG")
  }
}

```

```

#Data frame accumulation
coords[[counter]] <- data.frame(
  Gene_id = n,
  CpGs = CpG_count,
  flanking_region = as.character(i)
)

#Counter
counter <- counter + 1

#Message for done

message(paste("Finished processing", n, "."))

}

}
cpgCounts <- do.call(rbind, coords)

#Final message

message("---- Processing complete!----")

return(cpgCounts)
}

```

Function documentation: The following contains an example using a manually input data frame, `genesTest`. It is essential to have the proper syntax of the genes' symbols for mapping purpose: ex. `RHO` is incorrect in this instance. `Rho` is correct.

There are three arguments the user needs to input:

- `df`: Identify the data containing the gene symbols
- `GeneIDs`: Which column contains the gene symbol IDs in the specified `df`. This needs to be in quotation marks.
- `flankRegion`: Specify the buffer regions around the TSS. This can be manually input or a vector.

Common errors: `start_coord` length is zero implies an improper gene name. I am working on making a more human-interpretable error code.

Child File

The analyses in this file work off the Methylation parent file containing the `countCpG` function. Please run all chunks on that file prior to proceeding here.

```

genes <- read.csv("C:/Users/kathe/Documents/Enke/Data/gene_cell-type_methylationInput.csv")

head(genes)

```

```
##      Gene Cell.type
## 1  Pde6h      cone
## 2   Arr3      cone
## 3 Opn1mw      cone
## 4  Cngb3      cone
## 5  Gnat2      cone
## 6  Pde6c      cone
```

CpG counter

```
Mm_CpG <- countCpG(genes, "Gene",
                   flankRegion = c(50,100,200,500,1000),
                   species = "Mm")
```

```
#Converting Gene ID to upper since human symbols use the full capital format.
genes$Gene <- toupper(genes$Gene)
```

```
Hs_CpG <- countCpG(genes,
                   GeneIDs = "Gene",
                   flankRegion = c(50, 100, 200, 500, 1000),
                   species = "Hs")
```

Plotting general distributions

Loading libraries These are all base CRAN libraries. Not Bioconductor.

```
libs <- c("tidyverse",
         "car",
         "patchwork",
         "lme4",
         "ggeffects")

sapply(libs, library, character.only = T)
```

```
## Warning: package 'ggeffects' was built under R version 4.5.3
```

Plotting per species

Setting the theme. This is to ensure all the plots are cohesive with in the final product.

```
#Creating a theme for the document.
theme_childFile <- function(){
  theme(
    #Legend manipulation
    legend.position = "inside",
    legend.position.inside = c(0.1,0.8),
    legend.box.background = element_rect(color = "black",
                                          linewidth = 1),
    legend.title = element_text(size = 14,
                                face = "bold"),
    legend.text = element_text(size = 11,
                               color= "black"),
```

```

#Panel manipulation
panel.grid = element_blank(),
panel.border = element_rect(color = "black",
                             linewidth = 1),
axis.ticks = element_line(linewidth = 1),
axis.ticks.length = unit(5, "pt"),
axis.minor.ticks.length = rel(0.5),
strip.background = element_rect(fill = "white", color = NA),
strip.text = element_text(size = 15,
                           face = "bold"),

#Text manipulation
axis.title = element_text(face = "bold",
                           size = 17),
axis.title.x = element_text(margin = margin(t = 7)),
axis.title.y = element_text(margin = margin(r = 10)),
axis.text = element_text(size = 11,
                          color = "black")
})

```

Mouse:

```

mouse.only <- ggplot() +
  geom_histogram(data = Mm_CpG, aes(CpGs), stat = "bin",
                bins = 20,
                fill = "darkgrey",
                color = "black",
                alpha = 0.5) +
  labs(x = "Number of CpGs",
       y = "Count",
       title = "Mouse") +
  theme_bw() +
  theme_childFile()

human.only <- ggplot() +
  geom_histogram(data = Hs_CpG, aes(CpGs),
                stat = "bin",
                bins = 20,
                fill = "orchid3",
                color = "black",
                alpha = 0.5) +
  labs(x = "Number of CpGs",
       y = "Count",
       title = "Human") +
  theme_bw() +
  theme_childFile()

combined <- ggplot() +
  geom_histogram(data = Mm_CpG, aes(CpGs), stat = "bin",
                bins = 20,
                fill = "darkgrey",
                color = "black",
                alpha = 0.5) +
  geom_histogram(data = Hs_CpG, aes(CpGs),

```

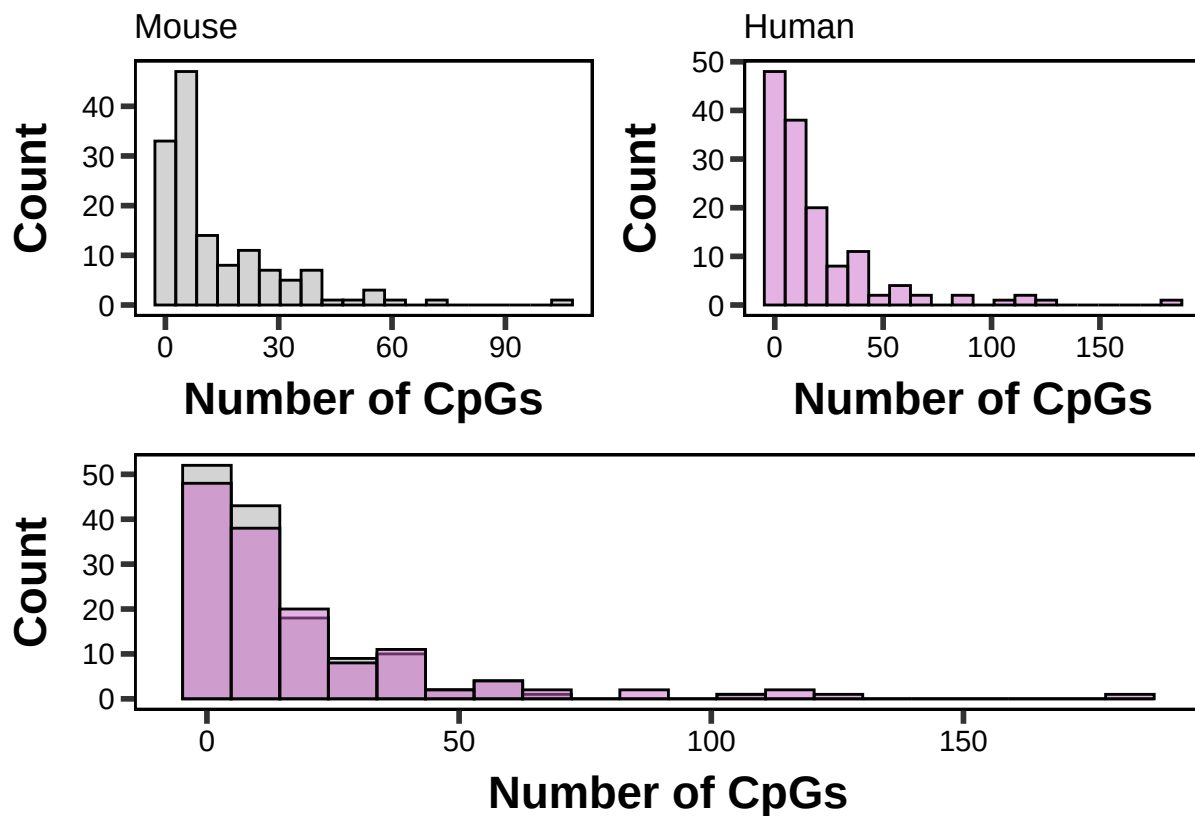
```

stat = "bin",
bins = 20,
fill = "orchid3",
color = "black",
alpha = 0.5) +
labs(x = "Number of CpGs",
y = "Count") +
theme_bw() +
theme_childFile()

full.histograms <- (mouse.only | human.only)/combined

full.histograms

```



```

#Creating CpG_total data frame to house the CpGs, organism, and cell-type
CpG_total <- Mm_CpG %>%
  mutate(
    Gene_id = toupper(Gene_id),
    organism = "Mm"
  ) %>%
  left_join(genes, by = join_by("Gene_id" == "Gene"))

#I did not do this in a great way. But it works.
Hs_temp <- Hs_CpG %>%

```

```

mutate(organism = "Hs") %>%
left_join(genes, by = join_by("Gene_id" == "Gene"))

CpG_total <- CpG_total %>%
  rbind(Hs_temp) %>%
  mutate(
    Cell.type = as.factor(Cell.type),
    organism = as.factor(organism),
    flanking_region = as.numeric(flanking_region),
    region = as.factor(flanking_region*2)
  )

rm(Hs_temp)
head(CpG_total)

```

```

##   Gene_id CpGs flanking_region organism Cell.type region
## 1  PDE6H    5           50      Mm     cone    100
## 2  PDE6H    5          100      Mm     cone    200
## 3  PDE6H    6          200      Mm     cone    400
## 4  PDE6H   21          500      Mm     cone   1000
## 5  PDE6H   38         1000      Mm     cone   2000
## 6   ARR3    0           50      Mm     cone    100

```

Visualizing both species distribution

Using a boxplot to look at the spread within cell-types and species.

```

ggplot(CpG_total, aes(region, CpGs)) +

  #Adding in boxplot
  geom_boxplot(aes(fill = organism),
               staplewidth = 0.5,
               width = 0.5,
               alpha = 0.7,
               outlier.shape = 20,
               position = position_dodge(width = 0.7)) +

  #Adding in points
  geom_point(aes(color = organism),
             alpha = 0.5,
             shape = 18,
             position = position_dodge(width = 0.7)) +
  facet_wrap(~Cell.type, labeller = as_labeller(c(
    "both" = "Both",
    "rod" = "Rod only",
    "cone" = "Cone only"
  ))) +
  labs(x = "Flanking region around TSS",
       y = "Number of CG dinucleotides",
       fill = "Organism",
       color = "Organism") +
  guides(y = guide_axis(

```

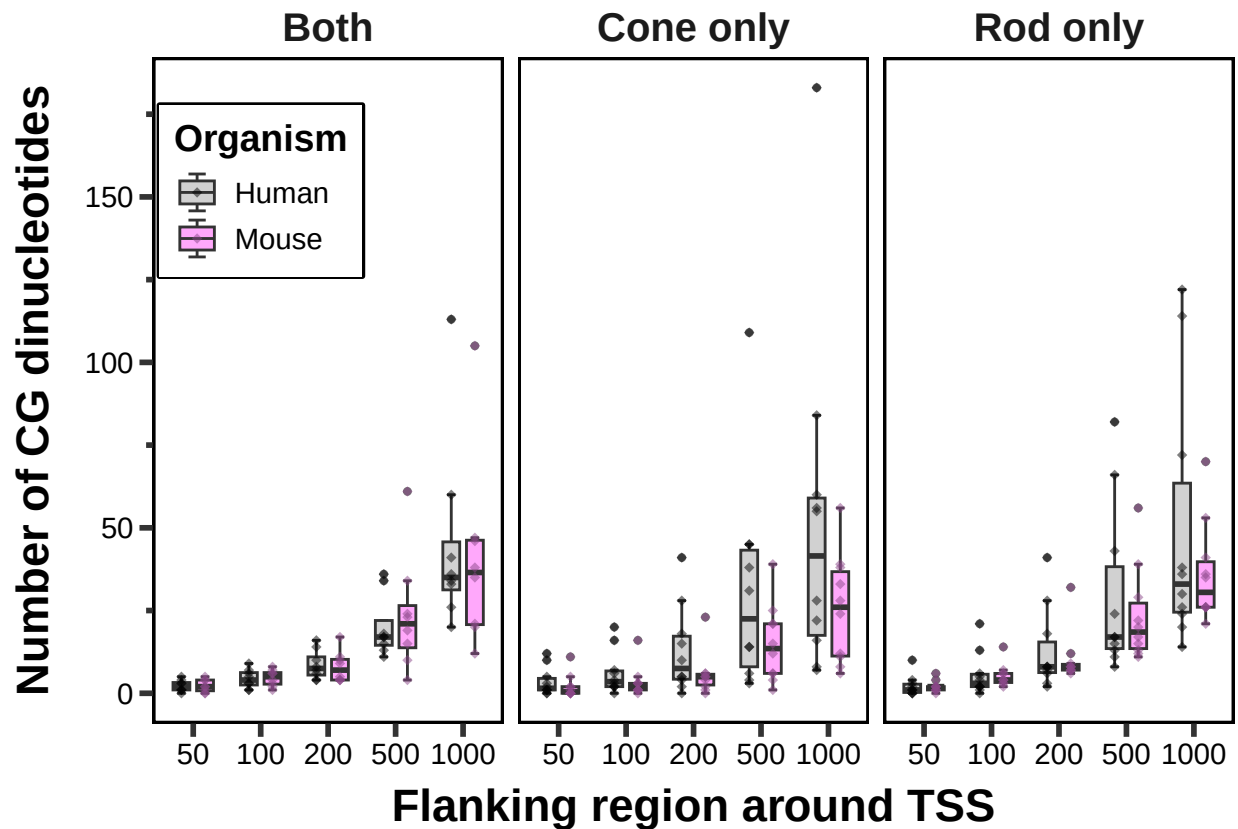


```

    minor.ticks = TRUE
  )) +
  scale_fill_manual(values = c("grey", "orchid1"),
                    labels = c(
                      "Hs" = "Human",
                      "Mm" = "Mouse"
                    )) +
  scale_color_manual(values = c("grey1", "orchid4"),
                     labels = c(
                       "Hs" = "Human",
                       "Mm" = "Mouse"
                     )) +

  #Renaming since flanking region is a double and region was a factor.
  scale_x_discrete(labels = c(
    "100" = "50",
    "200" = "100",
    "400" = "200",
    "1000" = "500",
    "2000" = "1000"
  )) +
  theme_bw() +
  theme_childFile()

```



```
#ggsave("CpG_Boxplots.png", height = 5, width = 7)
```

Plotting each gene

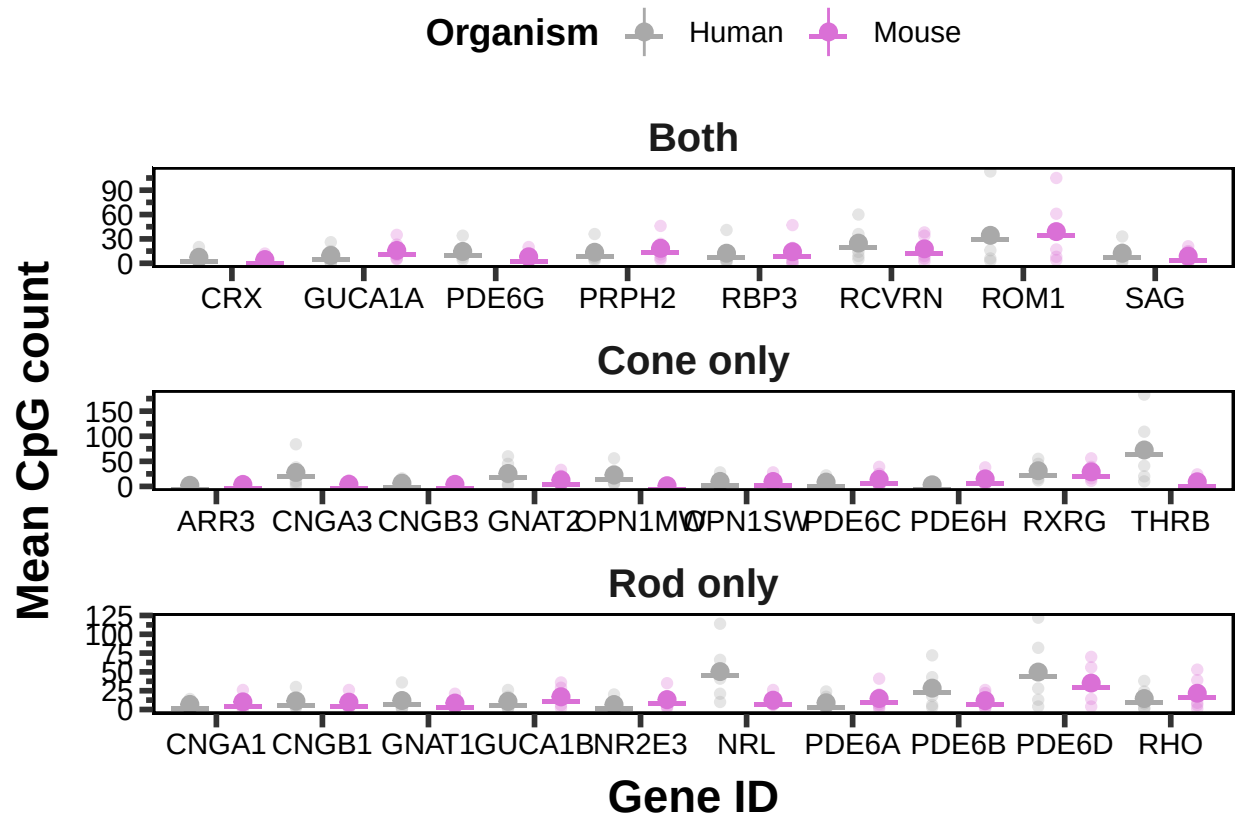
Plotting to see the distribution within each gene

```
ggplot(CpG_total, aes(Gene_id, CpGs, color = organism)) +
  geom_point(position = position_dodge(width = 1),
            alpha = 0.3) +
  stat_summary(fun = "mean",
              position = position_dodge(width = 1)) +
  stat_summary(fun = "mean",
              geom = "point",
              shape = 95,
              size = 8,
              position = position_dodge(width=1)) +
  facet_wrap(~Cell.type,
            nrow = 3,
            scales = "free",
            labeller = as_labeller(c(
              "both" = "Both",
              "rod" = "Rod only",
              "cone" = "Cone only")))) +
  scale_color_manual(values = c("darkgrey", "orchid"),
                    labels = c(
                      "Hs" = "Human",
                      "Mm" = "Mouse"
                    )) +
  labs(y = "Mean CpG count",
       x = "Gene ID",
       color = "Organism") +
  guides(y = guide_axis(minor.ticks = TRUE)) +
  theme_bw() +
  theme(
    #Legend manipulation
    legend.position = "top",
    legend.title = element_text(size = 14,
                                face = "bold"),
    legend.text = element_text(size = 11,
                                color = "black"),
    #Panel manipulation
    panel.grid = element_blank(),
    panel.border = element_rect(color = "black",
                                linewidth = 1),
    axis.ticks = element_line(linewidth = 1),
    axis.ticks.length = unit(5, "pt"),
    axis.minor.ticks.length = rel(0.5),
    strip.background = element_rect(fill = "white", color = NA),
    strip.text = element_text(size = 15,
                              face = "bold"),
    #Text manipulation
    axis.title = element_text(face = "bold",
```

```

        size = 17),
axis.title.x = element_text(margin = margin(t = 7)),
axis.title.y = element_text(margin = margin(r = 10)),
axis.text = element_text(size = 11,
                          color = "black")
)

```



```
#ggsave("CpG_GenePlot.png", height = 9, width = 9)
```

Data analysis

I am doing a GLM with a poisson model and an ANOVA with standardized CpG counts. This is due to the left-skewed data as well as it being count data with a high dispersion.

GLM

```

cpg.glm <- glmer.nb(CpGs ~ Cell.type * organism + (1|Gene_id),
  offset = log(flanking_region * 2),
  data = CpG_total)

summary(cpg.glm)

```

```
## Generalized linear mixed model fit by maximum likelihood (Laplace
## Approximation) [glmerMod]
## Family: Negative Binomial(5.9787) ( log )
## Formula: CpGs ~ Cell.type * organism + (1 | Gene_id)
## Data: CpG_total
## Offset: log(flanking_region * 2)
##
##          AIC          BIC      logLik -2*log(L)  df.resid
##      1722.6      1751.7      -853.3    1706.6      272
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -1.8968 -0.6896 -0.0652  0.5946  3.3106
##
## Random effects:
## Groups Name          Variance Std.Dev.
## Gene_id (Intercept) 0.3375    0.581
## Number of obs: 280, groups: Gene_id, 28
##
## Fixed effects:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)      -3.93389    0.22290 -17.649 < 2e-16 ***
## Cell.typecone       0.14088    0.29949   0.470 0.638063
## Cell.typerod        0.12081    0.29880   0.404 0.685974
## organismMm         -0.02716    0.12257  -0.222 0.824621
## Cell.typecone:organismMm -0.56368    0.17027  -3.311 0.000931 ***
## Cell.typerod:organismMm -0.02653    0.16417  -0.162 0.871619
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##              (Intr) C1l.typc C1l.typr orgnsM C1l.typc:M
## Cell.typecn -0.744
## Cell.typerd -0.746 0.555
## organismMm -0.272 0.202 0.203
## C1l.typcn:M 0.195 -0.270 -0.146 -0.720
## C1l.typrd:M 0.203 -0.151 -0.272 -0.747 0.538
```

Plotting GLMM

```
#Creating a predicted response for the GLMM
predict.2 <- predict_response(cpg.glm, terms = c("Cell.type", "organism"),
                             condition = list(flanking_region = 0.5))
```

```
## You are calculating adjusted predictions on the population-level (i.e.
## 'type = "fixed"') for a *generalized* linear mixed model.
## This may produce biased estimates due to Jensen's inequality. Consider
## setting 'bias_correction = TRUE' to correct for this bias.
## See also the documentation of the 'bias_correction' argument.
```

```
predict.2
```

```
## # Predicted counts of CpGs
##
## organism: Hs
##
## Cell.type | Predicted |      95% CI
## -----
## both      |      0.02 | 0.02, 0.02
## cone      |      0.02 | 0.02, 0.02
## rod       |      0.02 | 0.02, 0.02
##
## organism: Mm
##
## Cell.type | Predicted |      95% CI
## -----
## both      |      0.02 | 0.02, 0.02
## cone      |      0.01 | 0.01, 0.01
## rod       |      0.02 | 0.02, 0.02
##
## Adjusted for:
## * Gene_id = 0 (population-level)
```

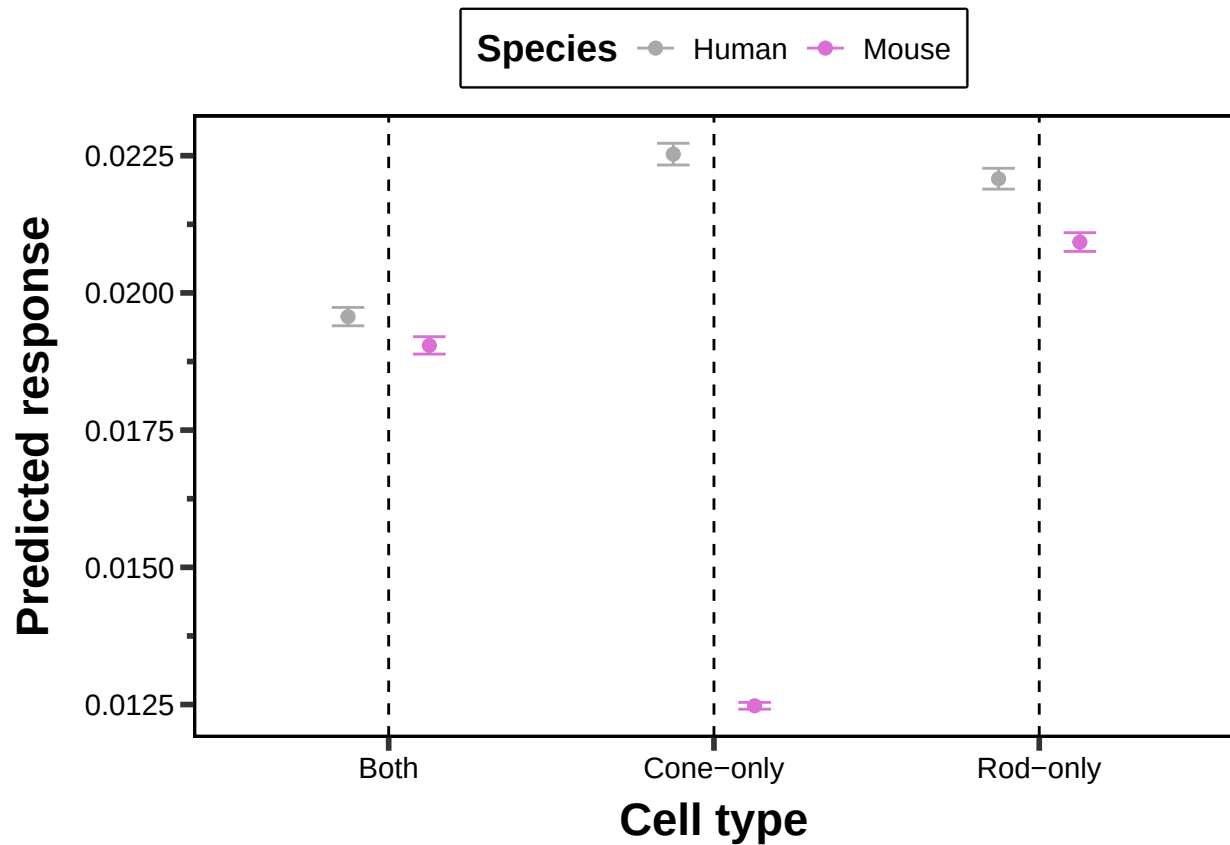
Plotting Predicting the number of CpGs using the NB GLMM created.

```
#Plotting the predicted variables
```

```
predict.Plot <- ggplot(predict.2, aes(x, predicted, color = group)) +
  geom_point(position = position_dodge(width = 0.5),
             size = 2) +
  geom_errorbar(aes(ymin = conf.low, ymax = conf.high),
               position = position_dodge(width = 0.5),
               width = 0.2) +
  scale_x_discrete(labels = c(
    both = "Both",
    cone = "Cone-only",
    rod = "Rod-only"
  )) +
  scale_color_manual(values = c("darkgrey", "orchid"),
                    labels = c(
                      "Hs" = "Human",
                      "Mm" = "Mouse"
                    )) +
  labs(y = "Predicted response",
       x = "Cell type",
       color = "Species") +
  guides(y = guide_axis(minor.ticks = TRUE)) +
  theme_bw() +
  theme_childFile() +
  theme(legend.position = "top",
        panel.grid.major.x = element_line(linetype = "dashed"),
        legend.background = element_rect(color = "white"),
```

```
linewidth = 0))
```

```
predict.Plot
```



```
#ggsave("C:/Users/kathe/Downloads/GenePlot.png")
```

```
#GLMM uses an offset with the flanking region which needs to be accounted for when plotting.
CpG.diff <- CpG_total %>%
  mutate(
    standardized = CpGs/(flanking_region * 2)
  )

#Creating the observed plot
observed.Plot <- ggplot(CpG.diff, aes(x = Cell.type, y = standardized, color = organism)) +
  stat_summary(fun = "mean", geom = "point",
    position = position_dodge(width = 0.5)) +
  stat_summary(fun.data = "mean_se", geom = "errorbar",
    position = position_dodge(width = 0.5),
    width = 0.2) +
  scale_x_discrete(labels = c(
    both = "Both",
```

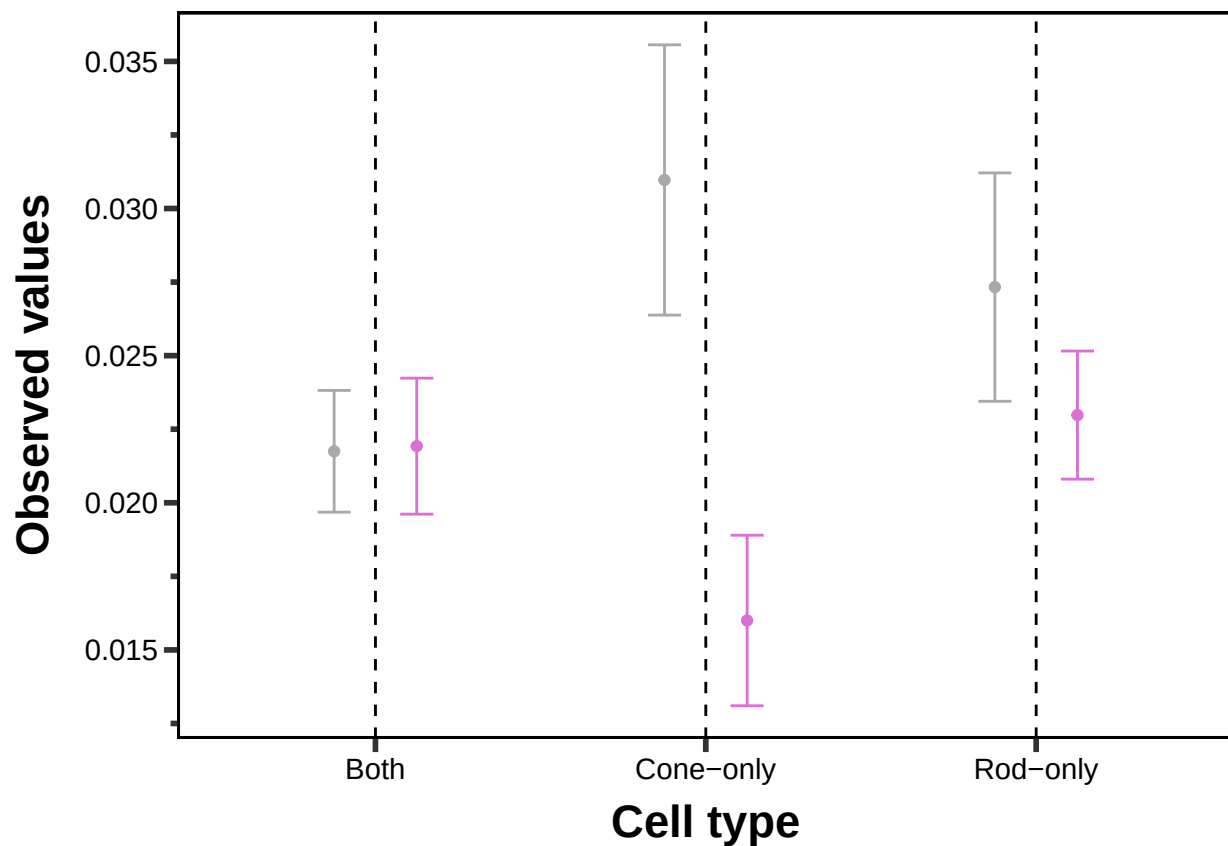
```

cone = "Cone-only",
rod = "Rod-only"
)) +
scale_color_manual(values = c("darkgrey", "orchid"),
                    labels = c(
                      "Hs" = "Human",
                      "Mm" = "Mouse"
                    )) +
labs(y = "Observed values",
     x = "Cell type",
     color = "Species") +
guides(y = guide_axis(minor.ticks = TRUE),
       color = "none") +
theme_bw() +
theme_childFile() +
theme(legend.position = "none",
      panel.grid.major.x = element_line(linetype = "dashed"),
      legend.background = element_rect(color = "white",
                                       linewidth = 0))

```

observed.Plot

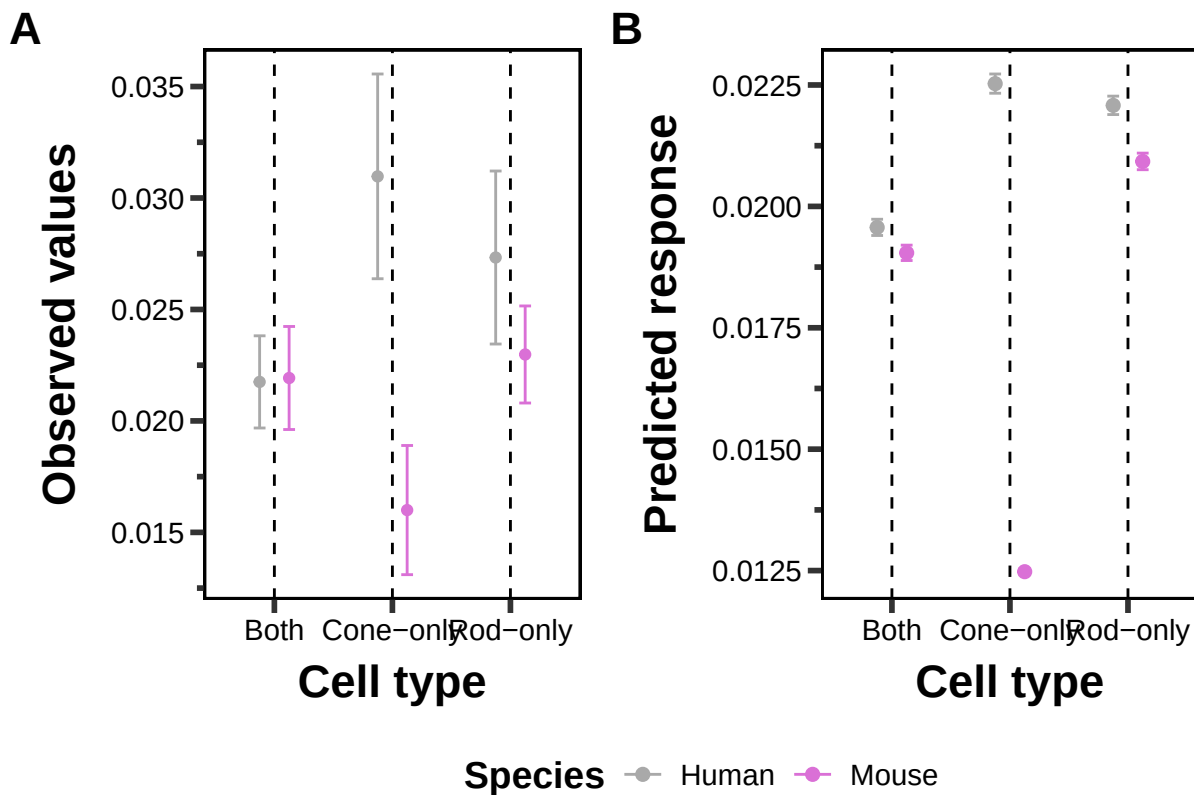
Creating a plot with the observed means



```
full.plot <- observed.Plot | predict.Plot

full.plot + plot_layout(guides = "collect") &
  plot_annotation(tag_levels = "A") &
  theme(legend.position = "bottom",
        plot.tag = element_text(face = "bold",
                                size = 17))
```

Combining plots for a full figure.



```
#ggsave("GLMM_CpG.plot.png", width = 8, height = 6)
```

Checking assumptions for GLMM Checking assumptions of the dispersion.

```
library(DHARMA)
```

```
## Warning: package 'DHARMA' was built under R version 4.5.3
```

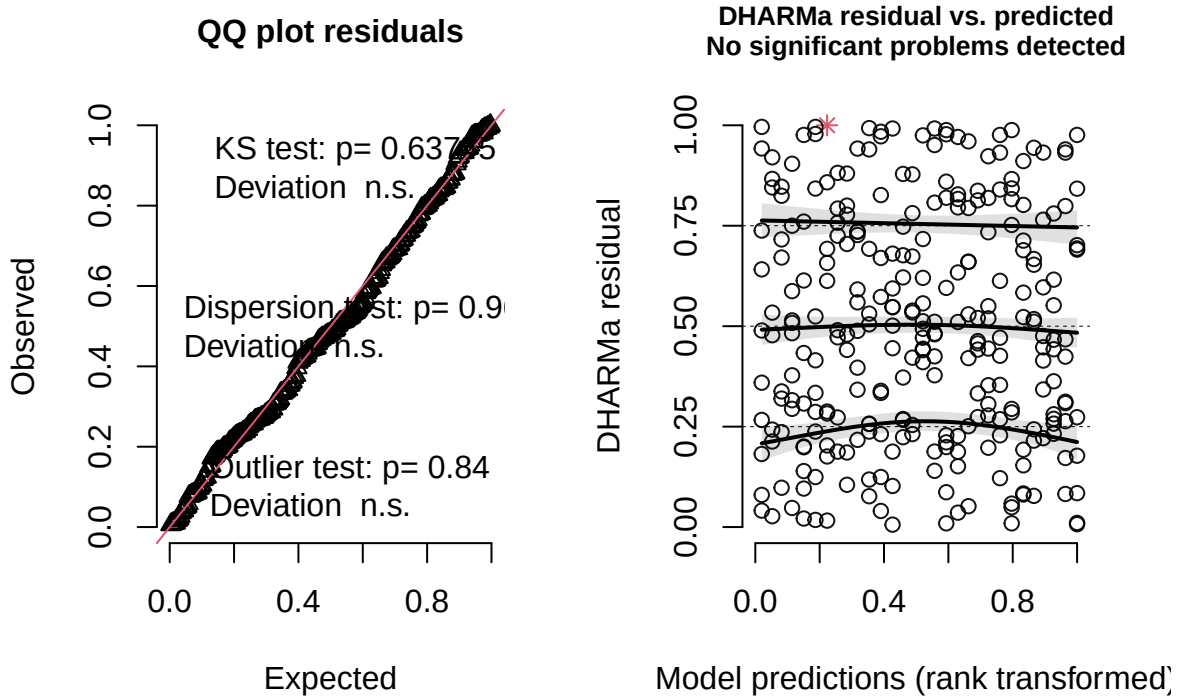
```
## This is DHARMA 0.4.7. For overview type '?DHARMA'. For recent changes, type news(package = 'DHARMA')
```



```
sim_res <- simulateResiduals(cpg.glm)
plot(sim_res)
```

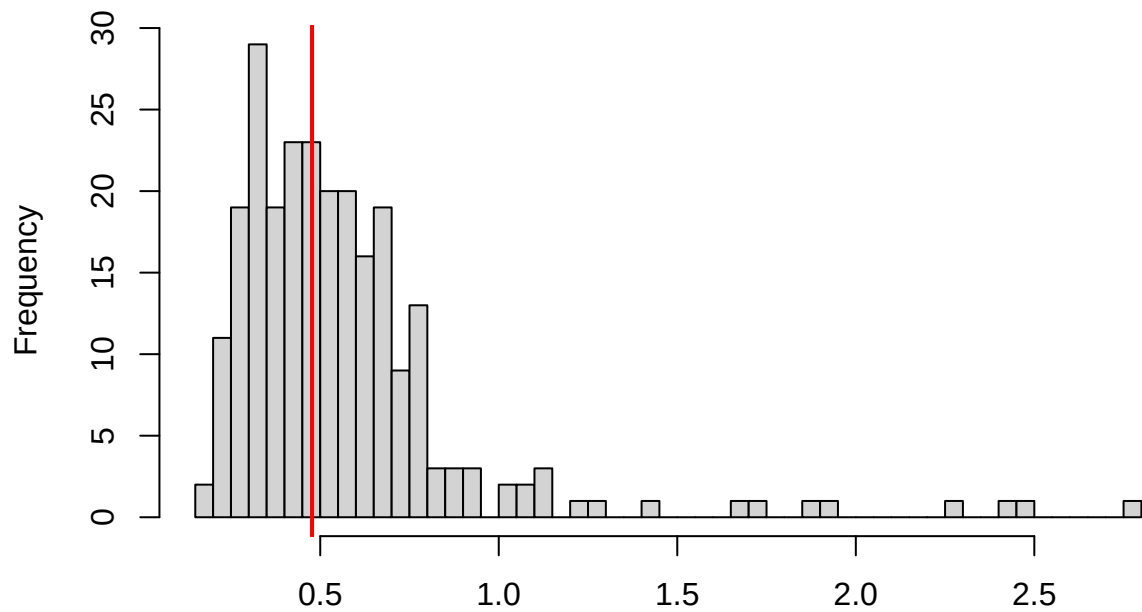
```
## Warning in newton(lsp = lsp, X = G$X, y = G$y, Eb = G$Eb, UrS = G$UrS, L = G$L,
## : Fitting terminated with step failure - check results carefully
```

DHARMA residual



```
testDispersion(sim_res)
```

DHARMA nonparametric dispersion test via sd of residuals fitted vs. simulated

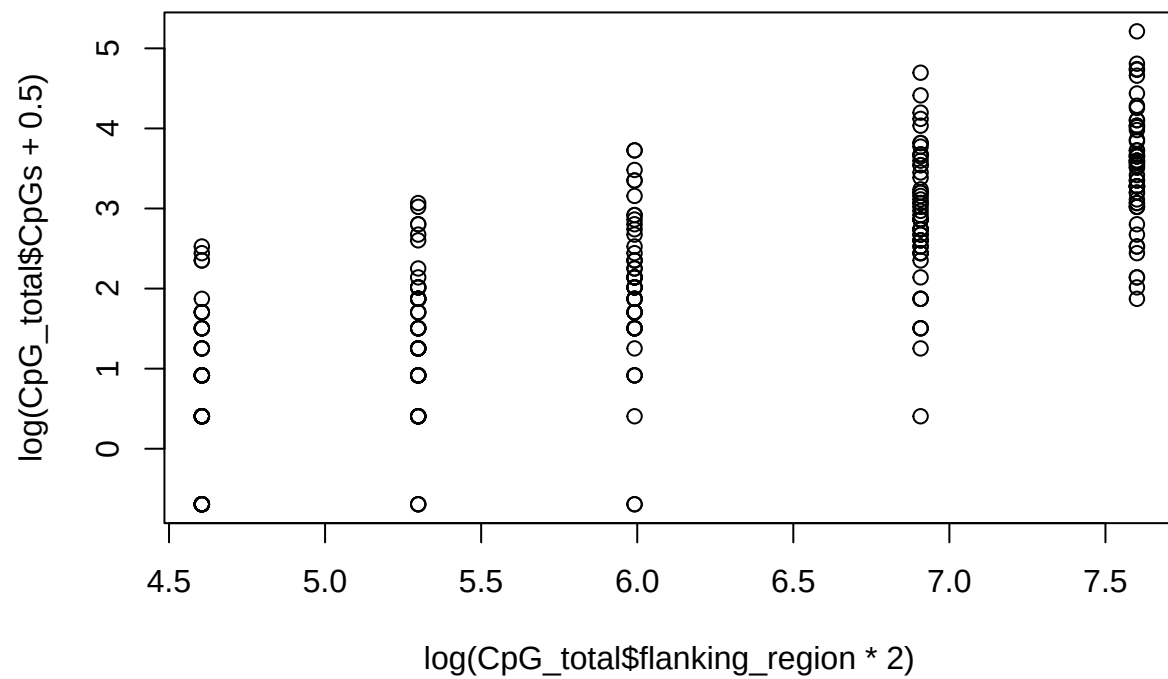


Simulated values, red line = fitted model. p-value (two.sided) = 0.96

```
##
## DHARMA nonparametric dispersion test via sd of residuals fitted vs.
## simulated
##
## data: simulationOutput
## dispersion = 0.83289, p-value = 0.96
## alternative hypothesis: two.sided
```

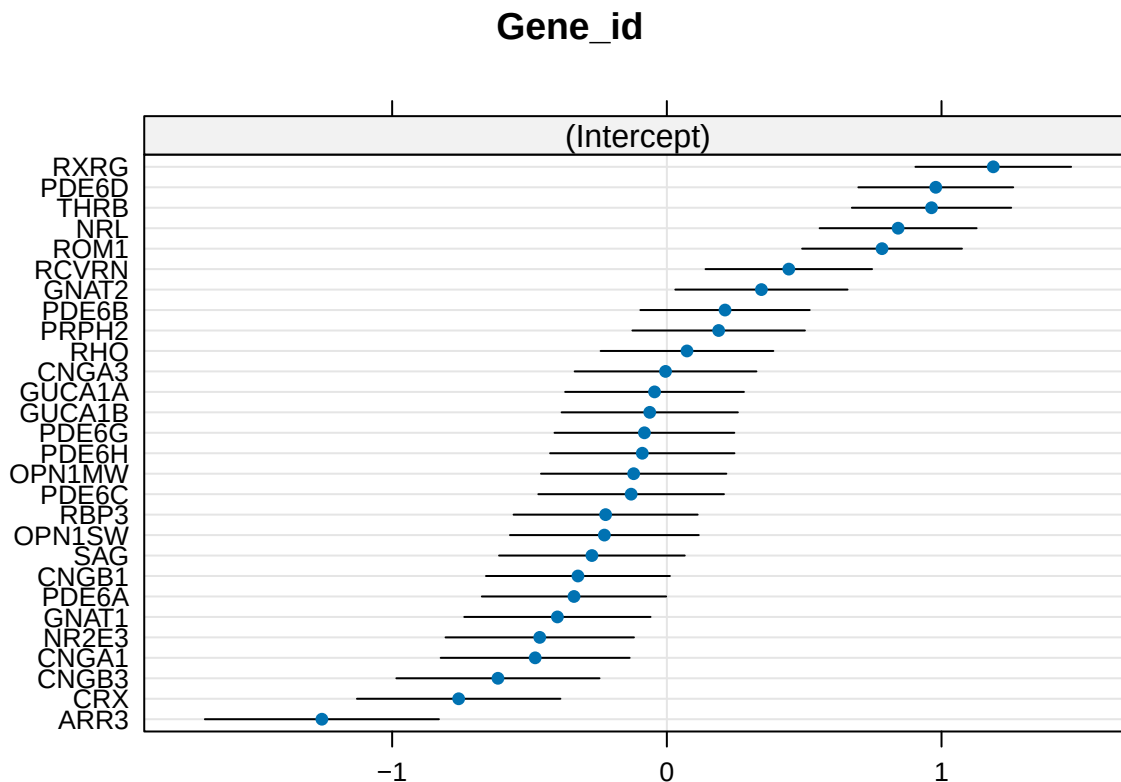
Checking the assumptions for the rest of it.

```
plot(log(CpG_total$flanking_region * 2), log(CpG_total$CpGs + 0.5))
```



```
#Should show roughly linear trend with slope ~1
library(lattice)
dotplot(ranef(cpg.glm, condVar = TRUE))
```

```
## $Gene_id
```



#Points should be roughly symmetrically distributed around 0

Looking at the exponential version of the coefficients for interpretation. The coefficients are in log format and need to be converted for proper analysis.

```
exp(coef(summary(cpg.glm)))
```

##	Estimate	Std. Error	z value	Pr(> z)
## (Intercept)	0.01956733	1.249690	2.163358e-08	1.000000
## Cell.typecone	1.15129032	1.349172	1.600649e+00	1.892810
## Cell.typerod	1.12841504	1.348246	1.498290e+00	1.985705
## organismMm	0.97320353	1.130398	8.012317e-01	2.281017
## Cell.typecone:organismMm	0.56910899	1.185625	3.649713e-02	1.000932
## Cell.typerod:organismMm	0.97381910	1.178410	8.507790e-01	2.390778

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
## LAPACK version 3.12.1
##
```

```

## locale:
## [1] LC_COLLATE=English_United States.utf8
## [2] LC_CTYPE=English_United States.utf8
## [3] LC_MONETARY=English_United States.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United States.utf8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats4      stats      graphics  grDevices utils      datasets  methods
## [8] base
##
## other attached packages:
## [1] lattice_0.22-7
## [2] DHARMA_0.4.7
## [3] ggeffects_2.3.2
## [4] lme4_2.0-1
## [5] Matrix_1.7-3
## [6] patchwork_1.3.2
## [7] car_3.1-5
## [8] carData_3.0-6
## [9] TxDb.Mmusculus.UCSC.mm39.knownGene_3.22.0
## [10] BSgenome.Mmusculus.UCSC.mm39_1.4.3
## [11] BSgenome.Hsapiens.UCSC.hg38_1.4.5
## [12] BSgenome_1.78.0
## [13] rtracklayer_1.70.1
## [14] BiocIO_1.20.0
## [15] Biostrings_2.78.0
## [16] XVector_0.50.0
## [17] GenomeInfoDb_1.46.2
## [18] org.Mm.eg.db_3.22.0
## [19] org.Hs.eg.db_3.22.0
## [20] biomaRt_2.66.1
## [21] GenomicFeatures_1.62.0
## [22] AnnotationDbi_1.72.0
## [23] Biobase_2.70.0
## [24] GenomicRanges_1.62.1
## [25] Seqinfo_1.0.0
## [26] IRanges_2.44.0
## [27] S4Vectors_0.48.0
## [28] BiocGenerics_0.56.0
## [29] generics_0.1.4
## [30] lubridate_1.9.5
## [31] forcats_1.0.1
## [32] stringr_1.6.0
## [33] dplyr_1.1.4
## [34] purrr_1.2.1
## [35] readr_2.1.6
## [36] tidyr_1.3.2
## [37] tibble_3.3.1
## [38] ggplot2_4.0.3
## [39] tidyverse_2.0.0

```

```

##
## loaded via a namespace (and not attached):
## [1] RColorBrewer_1.1-3          rstudioapi_0.18.0
## [3] jsonlite_2.0.0             datawizard_1.3.1
## [5] magrittr_2.0.4             farver_2.1.2
## [7] nloptr_2.2.1               rmarkdown_2.31
## [9] vctrs_0.7.1               memoise_2.0.1
## [11] minqa_1.2.8                Rsamtools_2.26.0
## [13] RCurl_1.98-1.17           tinytex_0.59
## [15] htmltools_0.5.9           S4Arrays_1.10.1
## [17] progress_1.2.3            curl_7.0.0
## [19] haven_2.5.5               SparseArray_1.10.8
## [21] Formula_1.2-5             plyr_1.8.9
## [23] httr2_1.2.2               cachem_1.1.0
## [25] GenomicAlignments_1.46.0  mime_0.13
## [27] iterators_1.0.14          lifecycle_1.0.5
## [29] pkgconfig_2.0.3           gap_1.14
## [31] R6_2.6.1                  fastmap_1.2.0
## [33] shiny_1.13.0              rbibutils_2.4.1
## [35] MatrixGenerics_1.22.0     digest_0.6.39
## [37] qgam_2.0.0                RSQLite_2.4.6
## [39] filelock_1.0.3            labeling_0.4.3
## [41] timechange_0.4.0          mgcv_1.9-3
## [43] httr_1.4.8                abind_1.4-8
## [45] compiler_4.5.0            doParallel_1.0.17
## [47] bit64_4.6.0-1             withr_3.0.2
## [49] S7_0.2.1                  BiocParallel_1.44.0
## [51] DBI_1.3.0                 MASS_7.3-65
## [53] rappdirs_0.3.4            DelayedArray_0.36.0
## [55] rjson_0.2.23              tools_4.5.0
## [57] otel_0.2.0                httpuv_1.6.17
## [59] glue_1.8.0                restfulr_0.0.16
## [61] promises_1.5.0            nlme_3.1-168
## [63] grid_4.5.0                gtable_0.3.6
## [65] tzdb_0.5.0                hms_1.1.4
## [67] xml2_1.5.2                foreach_1.5.2
## [69] pillar_1.11.1             later_1.4.8
## [71] splines_4.5.0             BiocFileCache_3.0.0
## [73] bit_4.6.0                 tidyselect_1.2.1
## [75] knitr_1.51                reformulas_0.4.4
## [77] SummarizedExperiment_1.40.0 xfun_0.56
## [79] matrixStats_1.5.0         stringi_1.8.7
## [81] UCSC.utils_1.6.1          yaml_2.3.12
## [83] boot_1.3-31               evaluate_1.0.5
## [85] codetools_0.2-20          cigarillo_1.0.0
## [87] BiocManager_1.30.27       cli_3.6.5
## [89] xtable_1.8-8              Rdpack_2.6.6
## [91] dichromat_2.0-0.1         Rcpp_1.1.1-1.1
## [93] dbplyr_2.5.2              png_0.1-8
## [95] XML_3.99-0.22             parallel_4.5.0
## [97] blob_1.3.0                prettyunits_1.2.0
## [99] gap.datasets_0.0.6        bitops_1.0-9
## [101] scales_1.4.0              insight_1.5.0
## [103] crayon_1.5.3              rlang_1.1.7

```

[105] KEGGREST_1.50.0